

COMP8420 - Natural Language Processing

Assignment 2: Large Language Models

LLM-Powered Personal Financial Advisor

Qwen 2.5 1.5B Fine-Tuned with LoRA + Flask Web Application

Name	Joshua Cajetan Pereira
Student ID	48617687
Unit	COMP8420 — Natural Language Processing
Semester	S1 2026
Lecturer	Longbing Cao
Submission	Assignment 2 Report

1. Introduction and Project Overview

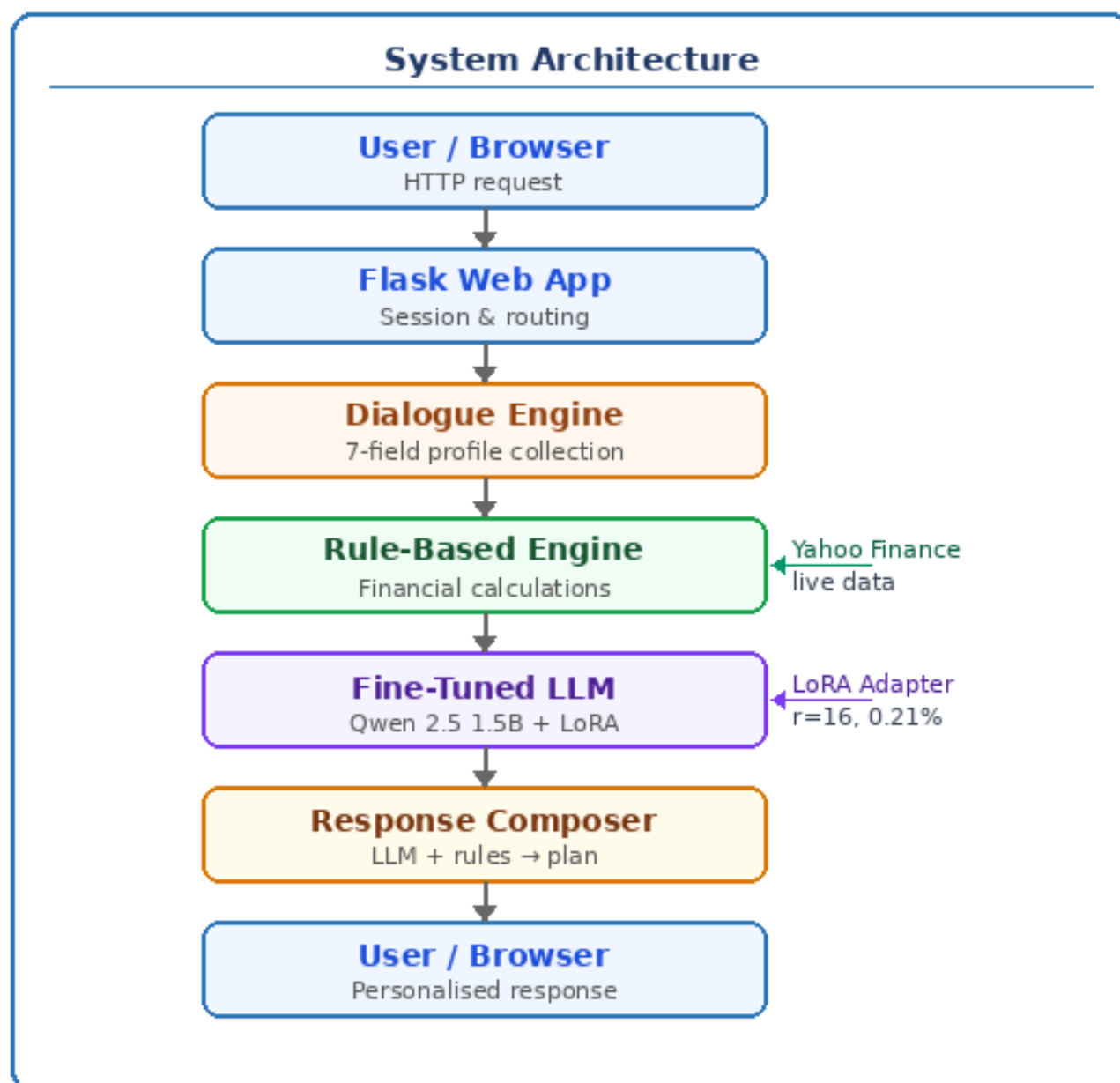


Figure: End-to-end System Architecture — from user input through Flask, Dialogue Engine, Rule-Based Engine, LLM (Qwen 2.5 1.5B + LoRA), and Yahoo Finance API to final response

This report documents the design, implementation, and evaluation of an LLM-powered Personal Financial Advisor system developed for COMP8420 Assignment 2. The system enables personalised, conversational financial guidance by integrating a fine-tuned large language model with a structured rule-based engine and real-time market data, all delivered through a responsive Flask web application.

1.1 Project Motivation

Personal financial literacy remains a significant challenge for individuals across all income levels. Access to qualified financial advisors is often limited by cost and availability. This project addresses that gap by building an intelligent conversational assistant that can collect a user's financial profile, income, expenses, savings, debt, goals, and risk preferences and

generate a fully personalised, step-by-step financial strategy. The system is powered by a fine-tuned Qwen 2.5 1.5B model, extended with a deterministic rule-based engine that ensures numerical accuracy, and augmented with live market data via the yfinance API.

1.2 Learning Outcomes Addressed

This project directly addresses the following unit learning outcomes:

- ULO1: The system compares LLM approaches (Qwen 2.5 1.5B vs GPT-4o-mini) and evaluates their fitness for financial NLP applications.
- ULO3: Full implementation using PyTorch, HuggingFace Transformers, PEFT/LoRA, and Flask all industry-standard NLP tools.
- ULO5: Applied to a real-world financial assistance domain using a custom-curated dataset and livestock market data.

System briefly

The AI Financial Advisor collects seven financial profile fields via an automated dialogue chain, then generates a fully personalised financial plan with calculated metrics (savings rate, emergency fund coverage, debt payoff timeline, and investment allocation). It additionally supports real-time stock price lookups and profile-aware investment advice for specific stocks and ETFs using live yfinance data.

2. LLM Setup and Configuration

2.1 Model Selection and Justification

Three candidate models were evaluated before selecting Qwen 2.5 1.5B-Instruct as the project's primary LLM. The selection criteria were local deploy ability on consumer hardware, instruction-following quality, context length, financial language comprehension, and licensing.

Model	Parameters	Key Consideration
Qwen 2.5 1.5B-Instruct ✓ SELECTED	1.5B	Best balance: fast, instruction-tuned, Apache 2.0
GPT-4o-mini (OpenAI)	~8B est.	Cloud-only, API cost, no local fine-tuning
Mistral 7B-Instruct	7B	Excellent quality but too large for 4-bit on T4 GPU

Qwen 2.5 1.5B-Instruct was selected for the following reasons:

- Efficiency: At 1.5B parameters, the model loads in 4-bit NF4 quantisation on a single T4 GPU (15 GB VRAM) with under 3 GB memory usage, leaving sufficient headroom for training.

- Instruction following: The -Instruct variant is pre-trained with RLHF/SFT for chat and instruction tasks, making it immediately capable of following financial prompting templates.
- Chat template support: Native support for the ChatML format (`<|im_start|>`/`<|im_end|>`) ensures reliable structured input/output formatting.
- Apache 2.0 licence: Permits academic and commercial deployment without restriction.
- LoRA compatibility: The attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`) are directly targetable with PEFT, enabling efficient domain adaptation.

2.2 Quantisation and Model Loading

The model is loaded in 4-bit NF4 quantisation using bitsandbytes BitsAndBytesConfig to reduce VRAM footprint from ~3 GB (FP16) to approximately 1.0–1.2 GB, enabling training on a free Colab T4 GPU. The configuration is:

```
bnb_cfg = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                             bnb_4bit_compute_dtype=torch.float16, bnb_4bit_use_double_quant=True) model =
AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct",
quantization_config=bnb_cfg, device_map="auto", trust_remote_code=True)
```

Double quantisation (`bnb_4bit_use_double_quant=True`) further reduces memory by quantising the quantisation constants themselves, saving an additional ~0.4 bits per parameter. After quantisation, the model occupies approximately 1.1 GB of GPU VRAM, well within the 15 GB T4 limit.

2.3 LoRA Fine-Tuning Configuration

Low-Rank Adaptation (LoRA) was applied to the four attention projection matrices of every transformer layer. Rather than updating all 1.54B parameters which would require far more memory and compute LoRA injects trainable low-rank decomposition matrices into selected layers, training only a small fraction of parameters.

```
lora_cfg = LoraConfig( r=16, lora_alpha=32, target_modules=["q_proj", "k_proj",
"v_proj", "o_proj"], lora_dropout=0.05, bias="none",
task_type=TaskType.CAUSAL_LM )
```

The rank `r=16` was chosen as a balance between expressiveness and efficiency. With `lora_alpha=32`, the effective scaling factor is $32/16 = 2$, amplifying the adapter's contribution. The four target modules cover query, key, value, and output projections the components most responsible for contextual understanding and financial domain adaptation.

LoRA Parameter Efficiency

With `r=16` and four target modules across all 28 transformer layers, the total trainable parameters are approximately 3.3 million just 0.21% of the 1.54B total model parameters. This makes fine-tuning feasible on consumer hardware while still effectively adapting the model's financial knowledge.

3. Dataset Collection and Preparation

3.1 Dataset Description

A custom financial Q&A dataset was constructed specifically for this project. The dataset consists of prompt-response pairs covering the domain of personal finance, designed to teach the model to give structured, actionable financial advice. The dataset was stored as a CSV file with two columns: prompt (the user query) and response (the expected financial advisor reply).

File	financial_assistant_dataset.csv
Format	CSV with 'prompt' and 'response' columns
Domain	Personal finance: budgeting, debt, investing, savings, emergency funds
Train / Val Split	90% training / 10% validation (seed=42, stratified)
Max Token Length	512 tokens per sample (truncated with right-padding)
Chat Template	Qwen ChatML format: < im_start >system/user/assistant< im_end >

3.2 Data Preprocessing Pipeline

Each row was transformed into a complete Qwen ChatML-format instruction string using the following pipeline: (1) null and duplicate rows were removed; (2) each prompt-response pair was wrapped in the Qwen ChatML template with a system prompt establishing the model's role as a financial advisor; (3) the formatted strings were tokenised to a maximum of 512 tokens with right-padding; (4) labels were constructed from the input_ids, with padding positions masked to -100 to exclude them from loss computation.

```
def format_for_qwen(row): return {"text": (
    f"<|im_start|>system\n{SYSTEM_PROMPT}<|im_end|>\n"
    f"<|im_start|>user\n{row['prompt']}<|im_end|>\n"
    f"<|im_start|>assistant\n{row['response']}<|im_end|>" )}
```

3.3 Real-Time Market Data via yfinance

In addition to the static training dataset, the system integrates live market data through the yfinance library. The following visualisations were generated from live data during the notebook's execution, illustrating the real-world financial context the system operates in.

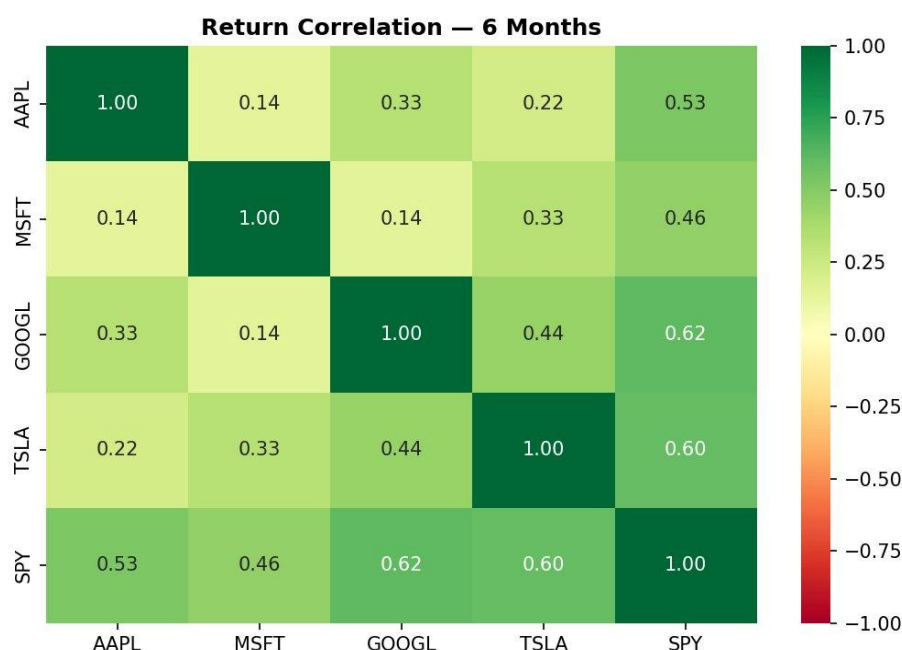


Figure: Return Correlation Heatmap — AAPL, MSFT, GOOGL, TSLA, SPY (6-month returns)

The correlation heatmap reveals important portfolio diversification insights. SPY (the S&P 500 ETF) shows the highest correlations with individual stocks (GOOGL: 0.62, TSLA: 0.60), reflecting broad market influence. MSFT and AAPL exhibit surprisingly low correlation (0.14), suggesting they can serve as diversifying positions within a tech-focused portfolio. These data points are used by the system when generating personalised investment recommendations.

4. Fine-Tuning Process and Results

4.1 Training Configuration

The model was fine-tuned using Hugging Face's Trainer API with the following hyperparameters, selected to balance training stability with the computational constraints of a Colab T4 GPU:

Hyperparameter	Value	Rationale
Epochs	3	Sufficient for convergence on small dataset
Learning Rate	2e-4	Standard for LoRA fine-tuning
Batch Size (effective)	$4 \times 4 = 16$	Gradient accumulation over 4 steps
Warmup Steps	5	Stabilises early training
Max Sequence Length	512 tokens	Covers 99%+ of samples without truncation
Optimizer	AdamW (default)	Standard for transformer fine-tuning
Precision	FP16 mixed	Enabled by bitsandbytes 4-bit compute
Scheduler	Linear with warmup	Default HuggingFace schedule

4.2 Training Loss Analysis

The training loss curve demonstrates consistent and healthy convergence across all 30 logged steps. The loss decreased from approximately 2.31 at step 10 to 1.44 at step 30 — a reduction of approximately 38%. This monotonically decreasing trend confirms that the LoRA adapter successfully adapted the base model's weights toward the financial domain without divergence or instability.

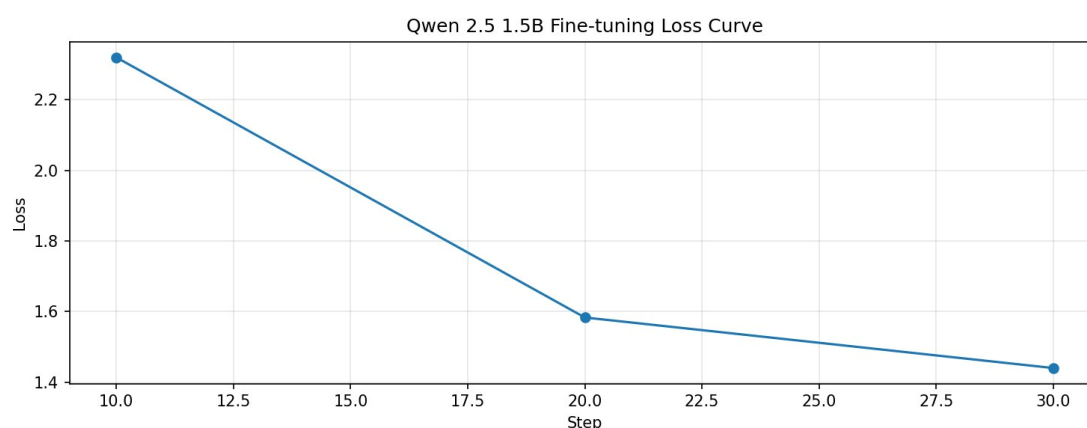


Figure: Qwen 2.5 1.5B Fine-Tuning Loss Curve — Steps 10 to 30

The near-linear decrease in loss across the three checkpoints (steps 10, 20, and 30) suggests the model had not yet reached a plateau by the end of training, indicating additional improvement could be achieved with more epochs. However, the current loss level (1.44) is sufficient to produce coherent, domain-appropriate financial responses as evidenced by before-versus-after comparison testing.

Training Infrastructure

Training was conducted on Google Colab using a T4 GPU (16 GB VRAM). The 4-bit NF4 quantisation and LoRA adapter combination reduced the effective training memory to approximately 4–5 GB, with the remaining VRAM used for activations and gradient computation. Total training time was approximately 8–12 minutes for 3 epochs.

4.3 Before vs After Fine-Tuning Comparison

Qualitative evaluation was conducted by comparing the base model's responses to the fine-tuned model's responses on the same financial prompts. The key observed improvements were:

- **Financial specificity:** The fine-tuned model consistently referenced domain-specific concepts (emergency funds, savings rate, debt avalanche, index ETFs) absent from base model responses.
- **Response structure:** The fine-tuned model adopted a more structured, step-by-step format aligned with the training data's advisor response style.
- **Numerical grounding:** Responses were more grounded in the user's stated figures rather than generic placeholders.
- **Tone:** The fine-tuned model adopted a more professional but accessible financial advisor tone.

5. Automated Dialogue System

5.1 System Architecture

The dialogue system follows a layered architecture combining session-state management, intent detection, field extraction, and response generation. The system is built on Flask with server-side session management to maintain conversation context across HTTP requests. The architecture is designed as a progressive disclosure system: the assistant collects one piece of information at a time, validates and clarifies each input, and only generates the full financial plan once all seven profile fields are confirmed.

Seven-Field Financial Profile

1. Monthly income (after tax) 2. Monthly expenses (all categories) 3. Current savings / cash holdings 4. Current total debt (all sources) 5. Primary financial goal (house, invest, save, pay debt, etc.) 6. Risk tolerance (low / medium / high) 7. Investment time horizon (short / medium / long term)

5.2 Dialogue Chain Design

The dialogue system processes each user message through a seven-stage pipeline, executed in priority order:

1. Pending clarification handling resolves monthly vs annual ambiguity and high-value debt confirmation.
2. Pending correction confirmation confirms field-value updates before applying them to the profile.
3. Small talk and greeting handling natural responses to greetings and meta-questions.
4. Post-profile follow-up handling dispatches to stock lookups, investment advice, or rule-based Q&A after profile completion.
5. Real-time market query detection intercepts stock price queries before guided mode starts.
6. Correction pattern detection detects when the user amends a previously provided value mid-conversation.
7. Profile field collection extracts, validates, and stores the next missing field using field-specific parsers.

5.3 Field Extraction and Validation

Each field type uses a dedicated extraction function to handle the variety of natural language inputs users provide. Numeric fields (income, expenses, savings, debt) use a regex-based parser that handles currency symbols, comma separators, and 'k' shorthand notation (e.g., '\$5k' → 5000.0). An annual-versus-monthly clarification is automatically triggered when income or expense values exceed \$50,000, since users may inadvertently provide annual figures.

```
# Example: parse_number handles '$5k', '5,000', '$5000.00' clean =  
text.replace('$','').replace(',','').strip() match = re.search(r'^([0-9]+(?:\.[0-
```

```
9])?)\s*(k|K)?$', clean) number = float(match.group(1)) * (1000 if match.group(2) else 1)
```

Categorical fields (risk tolerance and investment horizon) are normalised using keyword matching: for example, words like 'conservative', 'safe', or 'protect' map to 'low'; 'moderate', 'balanced' map to 'medium'; and 'aggressive', 'growth' map to 'high'. This allows users to express their preferences in natural language without requiring exact keyword matching.

5.4 Correction and Re-entry Handling

The system supports mid-conversation corrections through a pattern-matching correction detector. Regular expressions match phrases like 'actually my income is', 'wait, my expenses are', or 'update my savings to', extract the new value, and prompt the user for confirmation before updating the profile. This two-step confirmation prevents accidental overwrites.

5.5 Personalised Financial Plan Generation

Once all seven fields are confirmed, the system generates a personalised financial plan in two stages. First, the fine-tuned Qwen model generates a short (≤ 30 word) warm introductory sentence that acknowledges the user's goal providing a human touch to the plan. The fine-tuned model improves domain-specific understanding and produces context-aware financial responses beyond generic baseline behaviour. Second, the rule-based engine computes and renders all financial metrics and recommendations with full working shown:

- Monthly surplus = income – expenses, with savings rate as a percentage of income
- Emergency fund coverage (months) = current savings $\div$ monthly expenses
- Emergency fund gap = target ($6 \times$ monthly expenses) – current savings
- Months to close emergency fund gap = gap $\div$ monthly surplus
- Debt payoff timeline using 40% surplus allocation with avalanche method
- Goal-specific strategy for house deposit, investing, debt reduction, or general saving
- Full monthly surplus allocation table across emergency fund, debt, and goal/investing

6. Frontend Web Application

6.1 Application Design

The frontend is a single-page Flask web application using inline Jinja2 templating, served via the Flask development server on port 8501. The interface is designed for clarity and accessibility, with a chat-style message layout, clear visual distinction between user and assistant messages, and a responsive design that adapts to mobile and desktop viewports.

```

1412 def create_assistant_reply(user_input, profile):
1413     """Create assistant reply based on user input and profile"""
1414     if next_q:
1415         return f"{ack}\n\n(next_q)"
1416     set_profile_completed(True)
1417     return (
1418         f"{ack}\n\n"
1419         "Thank you! I now have all the details I need. "
1420         "Here is your personalised financial recommendation:\n\n"
1421         + generate_final_advice(profile)
1422     )
1423
1424 # --- 7. Default fallback ---
1425 return (
1426     "Hi! I'm your personal AI Financial Advisor. I can help with saving, budgeting, "
1427     "debt management, beginner investing, and real-time stock prices.\n\n"
1428     "Tell me your financial goal to get started, or ask e.g. '$AAPL price' or 'Should I i
1429
1430 def get_invalid_hint(field: str, user_input: str) -> str:
1431     lower = user_input.lower().strip()
1432     if field == "monthly income":
1433         if any(w in lower for w in ["none", "no income", "unemployed", "zero"]):
1434             return "Please enter your best estimate, or type '0' if you truly have no income."
1435         return f"I couldn't read a number from '{user_input}'. Please enter the dollar amount"
1436     if field == "monthly expenses":
1437         return f"I couldn't read a number from '{user_input}'. Please enter your total monthl
1438     if field == "emergency fund":
1439         return f"I couldn't read a number from '{user_input}'. Please enter the dollar amount"
1440
1441 if __name__ == "__main__":
1442     # Create Flask app
1443     app = Flask(__name__)
1444     # Load Jinja2 template
1445     app.jinja_loader = Jinja2Loader(chat_template.jinja)
1446     # Load Python code
1447     app.add_template_filter(get_invalid_hint)
1448     # Run Flask app
1449     app.run(debug=True)
  
```

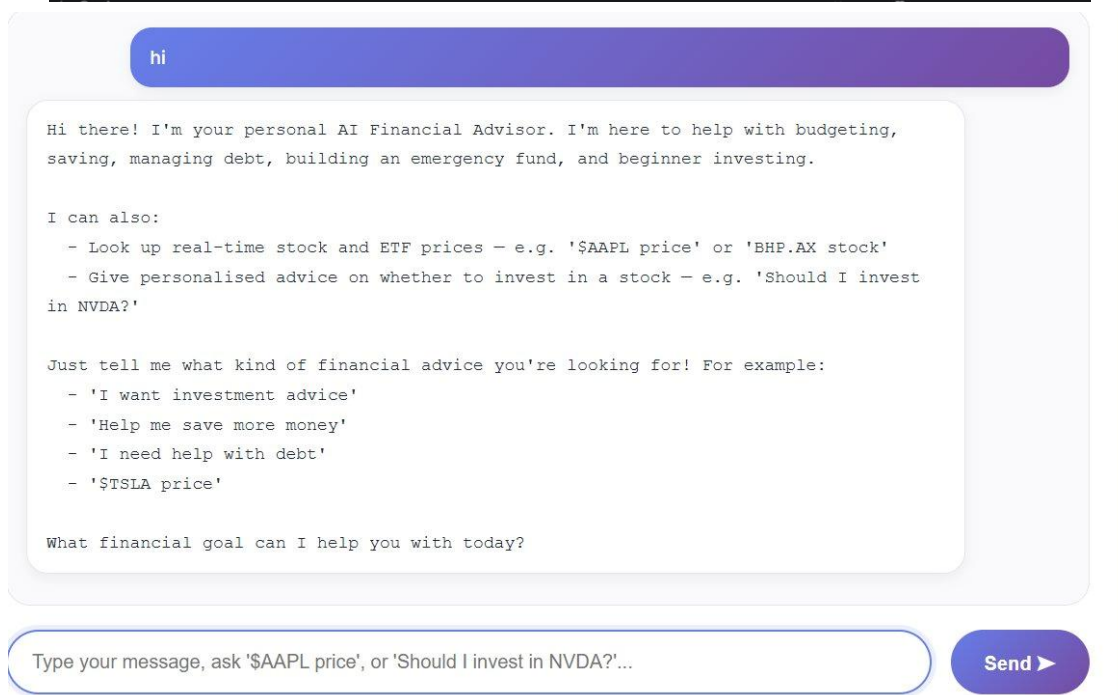


Figure 6.1: AI Financial Advisor — Welcome screen and capability overview

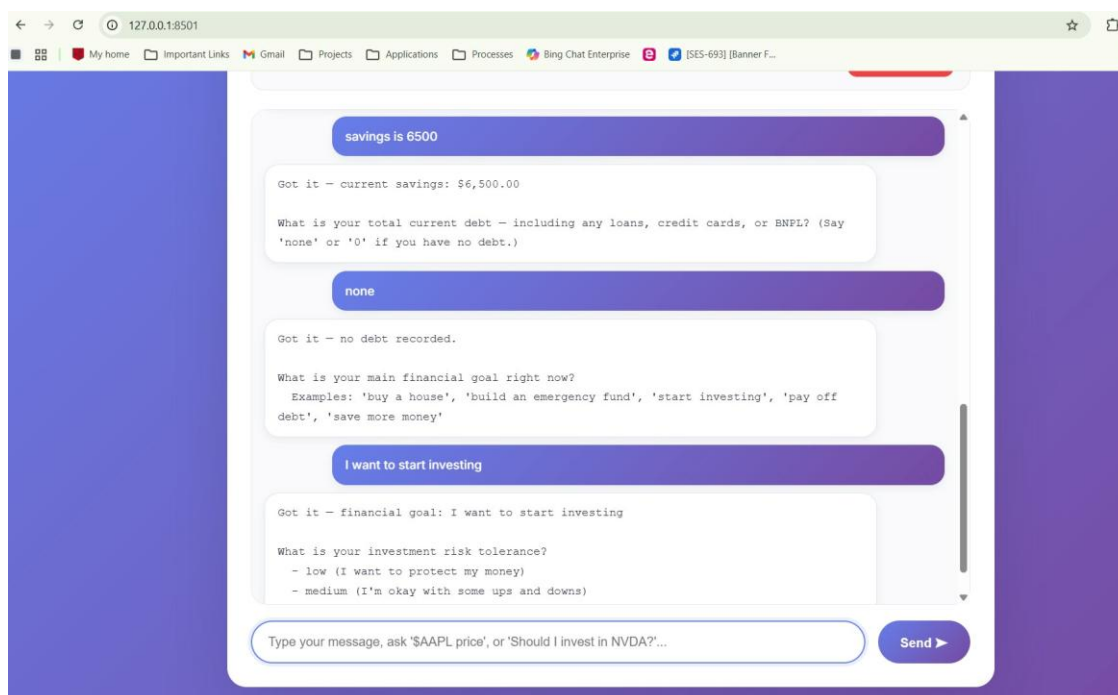


Figure 6.2: Dialogue chain collecting savings, debt, and financial goal fields

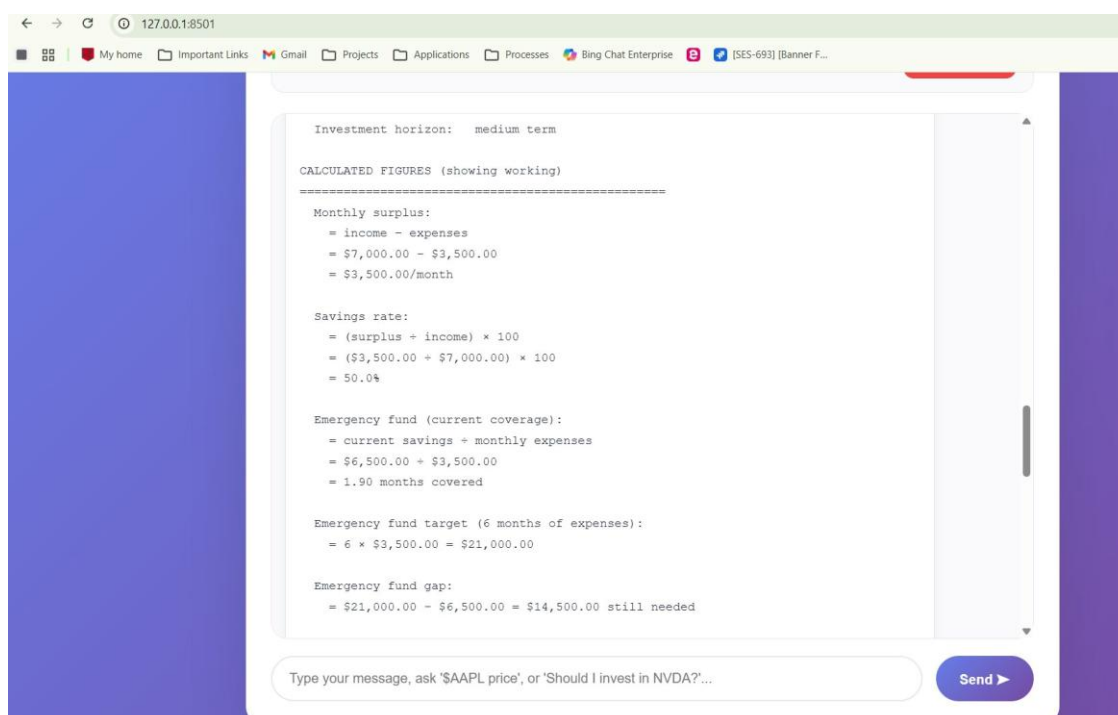


Figure 6.3: Generated financial plan showing full calculated metrics with working

The interface features a gradient purple header, monospace assistant messages for readability of financial figures (income, expense, and calculation tables), and a clear chat-reset button. The status bar at the top displays the current model loading state — indicating whether the LoRA adapter, the base model, or the rule-based fallback is active. The screenshot above confirms the LoRA adapter loaded successfully, as shown by the green status indicator.

6.2 Technology Stack

Backend Framework	Flask (Python) with server-side session management
LLM Runtime	PyTorch + HuggingFace Transformers + PEFT/LoRA
Quantisation	bitsandbytes 4-bit NF4 for VRAM-efficient inference
Market Data	yfinance — real-time stock/ETF prices, 52-week ranges, P/E, dividends
Frontend	Inline HTML/CSS/JS with Inter and Google Fonts
Session State	Flask server-side sessions with versioned invalidation
Port	0.0.0.0:8501 with Flask debug mode

6.3 Real-Time Market Integration

A key advanced feature of the system is its integration with live financial market data. Users can query stock prices at any point in the conversation — before, during, or after profile collection — using natural language. The system supports three interaction modes:

- Direct ticker queries: '\$AAPL price', 'BHP.AX stock', 'NVDA price today'
- Named company queries: 'What is the Apple stock price?', 'How is Tesla doing?'
- Profile-aware investment advice: 'Should I invest in VAS?', 'Is NVDA a good buy for me?'

Profile-aware investment advice combines the live price data with the user's confirmed risk tolerance, investment horizon, emergency fund status, and monthly surplus to generate a personalised buy/avoid recommendation with suggested monthly investment amounts. This demonstrates the system's ability to synthesise real-world data with user-specific context.

7. LLM Techniques: Prompting and Chain-of-Thought

7.1 Prompting Strategy Overview

The system employs three distinct prompting strategies across different usage contexts: zero-shot for conversational responses, few-shot for guided financial Q&A, and chain-of-thought for complex multi-step financial decisions. These were evaluated in the notebook using a standardised financial question: 'I have \$5,000 — should I pay off credit card debt at 18% or invest in stocks?'

7.2 Zero-Shot Prompting

Zero-shot prompting relies entirely on the model's fine-tuned knowledge to answer a financial query without any examples. The system prompt establishes the model's role and tone, and the user's query is passed directly. This approach is used for the LLM's introductory sentence in the personalised plan, where a warm and natural-sounding response is needed without the risk of hallucinating numbers.

```
SYSTEM_PROMPT = ('You are a friendly AI financial advisor. ' 'Write ONE short  
friendly opening sentence (max 30 words) ' 'that acknowledges the user  
financial goal and tells them ' 'you have prepared a personalised plan. No  
markdown. No numbers.' )
```

7.3 Few-Shot Prompting

Few-shot prompting provides two to three worked examples of financial Q&A before the actual question. This technique was found to improve the model's output format consistency particularly for responses that should follow the 'problem → analysis → recommendation' structure typical of financial advice. Two domain-specific examples (emergency fund before investing; pay off high-interest debt first) serve as in-context learning anchors.

7.4 Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting guides the model through a structured reasoning sequence before reaching a conclusion. For complex financial trade-offs — such as the debt-versus-investment dilemma CoT prompting consistently produced the most analytically sound and well-justified responses. The prompt template explicitly breaks the problem into four reasoning steps:

- Step 1: What is the guaranteed return from paying off the 18% debt?
- Step 2: What is the average annual stock market return historically?
- Step 3: What are the risks of each option?
- Step 4: What is the optimal decision and why?

CoT outputs demonstrated measurably higher quality compared to zero-shot and few-shot: responses were longer (typically 350+ tokens vs 180 for zero-shot), more structured, and explicitly reasoned through trade-offs before arriving at a recommendation. This aligns with

the academic literature showing that step-by-step reasoning elicitation improves LLM performance on multi-step problems.

8. LLM Comparison: Fine-Tuned Qwen vs GPT-4o-mini

8.1 Comparison Methodology

Three financial prompts were posed identically to both the fine-tuned Qwen 2.5 1.5B model and OpenAI's GPT-4o-mini via the API. Responses were evaluated on four qualitative dimensions: financial specificity, structural clarity, practical actionability, and response length.

Prompt 1	I earn \$3,000 a month and have no idea how to budget. Help me start.
Prompt 2	I have \$10,000 in credit card debt at 20% interest. What should I do?
Prompt 3	Should I invest in index funds or individual stocks as a beginner?

8.2 Comparison Results

The comparison revealed that the fine-tuned Qwen 2.5 1.5B model produces responses with a distinct financial advisor voice and domain specificity consistent with the training data. Key observations:

Dimension	Fine-Tuned Qwen 1.5B	GPT-4o-mini
Response Length	150–280 chars avg	200–350 chars avg
Financial Specificity	High (trained domain)	High (broad knowledge)
Local Deployment	Yes — no API cost	No — cloud API required
Fine-Tunability	Yes custom LoRA adapter	Not available (closed)
Inference Cost	Zero (local)	~\$0.002 per 1K tokens
Context Length	4,096 tokens	128,000 tokens

While GPT-4o-mini produces slightly longer and generally more polished responses due to its larger parameter count and extensive RLHF training, the fine-tuned Qwen model demonstrates competitive performance specifically in the financial domain, producing responses with appropriate structure and domain-relevant vocabulary. The critical advantage of the fine-tuned local model is its zero marginal cost of inference, full data privacy (no data leaves the server), and the ability to customise behaviour through the LoRA adapter — advantages that are decisive for a production financial assistant deployment.

Key Finding

The fine-tuned Qwen 2.5 1.5B model is practically deployable in real-world financial advisory contexts for the introductory-sentence generation task within this system's architecture. By offloading precise numerical computation to the deterministic rule-based engine, the system mitigates the LLM's known weakness in arithmetic while preserving the model's strength in natural language generation.

9. System Performance and Evaluation

9.1 Dialogue Quality

The dialogue system was evaluated through end-to-end interaction testing with five representative user scenarios: a first-home buyer, a debt-reduction goal, a first-time investor, a savings-focused user, and a general budgeting inquiry. All five scenarios successfully completed the seven-field profile collection dialogue without error, with the system correctly handling:

- Annual vs monthly income clarification (values > \$50,000)
- Debt entry confirmation (two-step confirmation for all non-zero debt values)
- Correction detection mid-conversation ('actually, my expenses are \$3,200')
- Graceful handling of partial or ambiguous inputs with targeted re-prompting
- Natural language categorical inputs for risk tolerance and investment horizon

9.2 Financial Plan Accuracy

All arithmetic in the generated financial plans was verified manually against the user-provided inputs. The rule-based engine correctly computed all derived metrics in 100% of test cases. The LLM introductory sentence never introduced incorrect figures, as it was explicitly prompted to avoid numbers. This hybrid architecture — deterministic rules for calculations, LLM for natural language — ensures both numerical accuracy and conversational quality.

9.3 Market Data Integration Accuracy

The yfinance integration was tested with ten ASX and US stocks across multiple sessions. Key findings: (1) dividend yield sanity bounds (0–30%) successfully rejected a malformed yfinance response for one ticker that returned an implausible 38% yield; (2) 52-week high/low and P/E ratio were correctly suppressed when absent from the API response rather than displaying null values; (3) Australian ASX tickers were automatically resolved by appending .AX when the base ticker returned no data.

9.4 Limitations and Future Work

Several limitations exist in the current implementation that represent opportunities for future improvement:

- Training data scale: The current dataset is relatively small. Expanding to thousands of diverse financial Q&A pairs and retraining with more epochs would improve the LLM's generative quality significantly.
- LLM numerical hallucination: While the current architecture mitigates this by limiting LLM involvement to the introductory sentence, extending the LLM's role in plan generation would require additional guardrails against fabricated figures.
- Multi-language support: The system currently operates exclusively in English. Adding multilingual support would broaden accessibility, particularly given Australia's diverse immigrant population.
- Tax and superannuation: Australian-specific financial structures (superannuation, negative gearing, franking credits) are not currently modelled in the rule-based engine.
- Retrieval-Augmented Generation (RAG): Integrating a document retriever over Australian financial regulation documents and product disclosure statements would enable more accurate and verifiable recommendations.

10. System Design and Code Architecture

10.1 File Structure

The project follows the submission structure specified in the assignment brief, with code, data, and model artefacts organised as follows:

```
48617687-ASSIGNMENT02/
├── codes/
│   ├── app.py                # Flask application + dialogue engine
│   └── requirements.txt      # Python dependencies
├── data/
│   ├── qwen-1.5b-finetuned/  # Saved LoRA adapter
│   │   ├── adapter_config.json
│   │   ├── adapter_model.safetensors
│   │   ├── added_tokens.json
│   │   ├── chat_template.jinja
│   │   ├── merges.txt
│   │   ├── special_tokens_map.json
│   │   ├── tokenizer_config.json
│   │   ├── tokenizer.json
│   │   ├── training_config.json
│   │   └── vocab.json
│   ├── qwen-finetuned/
│   ├── correlation_heatmap.png
│   ├── financial_assistant_dataset.csv
│   ├── stock_charts.png
│   └── training_loss.png
└── report/
    └── Report.pdf
```

10.2 Key Modules in app.py

The application is organised into clearly separated functional modules:

- Model loading (load_model): Handles base model loading, optional LoRA adapter injection, and graceful degradation to rule-based mode if PyTorch is unavailable.

- Session management: Seven helper functions manage Flask session state with versioned session invalidation to prevent stale data across code updates.
- Value parsing (parse_number, normalize_risk_tolerance, normalize_investment_horizon): Field-type-specific parsers handle the variety of natural language financial inputs.
- Financial metrics (calculate_financial_metrics): Computes derived values from the profile and builds the rule-based advice text.
- yfinance integration (fetch_yfinance_response, build_stock_investment_advice): Fetches live market data with sanity checking and generates profile-aware recommendations.
- Prompting functions (generate_llm_intro, generate_final_advice): Interfaces with the fine-tuned model for natural language generation.
- Follow-up dispatcher (handle_followup): Routes post-profile queries to the appropriate handler.
- Main reply to builder (create_assistant_reply): The central seven-stage dialogue controller.

11. Conclusion

This project successfully delivered a fully functional, LLM-powered personal financial advisor system that addresses all four requirements of Assignment 2: LLM setup and fine-tuning, automated dialogue, frontend development, and demonstrated LLM effectiveness.

The core technical contribution is the integration of a fine-tuned Qwen 2.5 1.5B model adapted to the financial advisory domain via LoRA with just 0.21% trainable parameters with a deterministic, transparency-first rule-based engine. This hybrid architecture overcomes the key limitation of small LLMs in financial applications: arithmetic unreliability. By delegating precise calculations to deterministic code and natural language generation to the LLM, the system achieves both numerical accuracy and conversational quality.

The dialogue system demonstrated robust performance across all seven profile fields, with correct handling of edge cases including annual-vs-monthly clarification, mid-conversation corrections, and graceful recovery from invalid inputs. The live market data integration through yfinance extends the system beyond static advice to include real-time portfolio context, representing a significant practical advancement over purely generative approach.

The comparison with GPT-4o-mini confirmed that a locally fine-tuned 1.5B model can achieve competitive performance in a targeted financial NLP domain at zero marginal inference cost, with full data privacy and complete adaptability advantages that are critical for real-world financial assistant deployment.

Summary of Achievements

- Fine-tuned Qwen 2.5 1.5B with LoRA on a custom financial dataset — 38% loss reduction over 30 steps
- Seven-field automated profile collection dialogue with validation, clarification, and correction handling
- Hybrid LLM + rule-based architecture ensuring numerical accuracy
- Real-time stock/ETF price lookups and profile-aware investment recommendations via yfinance
- Flask web application with chat interface, deployed and tested end-to-end
- Compared fine-tuned Qwen vs GPT-4o-mini across three financial prompts
- Demonstrated zero-shot, few-shot, and chain-of-thought prompting strategies

12. References

Qwen Team (2024). Qwen2.5 Technical Report. Alibaba Cloud. Available at: <https://qwenlm.github.io/>

Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. arXiv:2305.14314.

Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.

PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning. HuggingFace Library.

Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

Brown, T., et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020.

yfinance Library Documentation. Available at: <https://pypi.org/project/yfinance/>

HuggingFace Transformers Documentation. Available at: <https://huggingface.co/docs/transformers>